

マジカル★観覧車の詳細設計：通信関連の構成

25G1041 菊池侑人

2026 年 5 月 18 日

目次

1	全体の仕様	2
2	通信機能の設計	3
2.1	全体の概要	3
2.2	無線通信仕様 (UDP)	4
2.3	物理同期信号の設計 (LED)	5
3	ソフトウェア設計	5
3.1	ファイル構成	5
3.2	オブジェクト設計	6
3.2.1	クラス図	6
4	処理フローの設計	6
4.1	LED を用いた演奏同期機能設計	6
4.2	回転速度同期機構の設計	8
4.3	演奏制御機能設計	9
5	テストの設計	10
5.1	BPM 追従テスト	10
5.2	同期精度テスト	10
5.3	負荷テスト	11

1 全体の仕様

本設計に使用する定数は表 1 の通りである。これらは実装において、プログラム内で `const` を使って定義する。

表 1 設計に利用する定数一覧.

型	名前	説明	所属	メモリ量 [byte]
float	VCC	Arduino への供給電圧 V_{cc}	Main	4
uint16_t	LOCAL_PORT	通信に利用するポート番号	SyncManager	2
char[]	SSID	Wi-Fi のネットワーク名	SyncManager	Max:33
char[]	PASS	その Wi-Fi のパスワード	SyncManager	Max:64
uint8_t	PIN_MOTOR_PWM	モーター制御用の出力ピン	MotorControl	1
uint8_t	bpmTable[]	5 つの離散値 BPM 格納	MotorControl	5

SSID および PASS のメモリ量について、参考文献 [1] によると、Arduino WiFi ライブラリでは SSID の最大長を 32 文字、Wi-Fi パスワード (WPA キー) の最大長を 63 文字として定義されている。また、Arduino 言語を含む C 言語や C++ の仕様として、文字列の終点を表すヌル文字 (`\0`) を格納するため 1 バイト分を加算する必要がある。そのため、本設計では SSID を 33 byte、PASS を 64 byte と算出した。

さらに、bpmTable[] の中身は表 2 のように定義されている。

表 2 bpmTable[] に格納する BPM 値.

インデックス番号	0	1	2	3	4
BPM 値	60	90	120	150	180

変数の仕様 本設計に使用する変数は表 3 の通りである。

表 3 設計に利用する変数一覧.

型	名前	説明	所属	メモリ量 [byte]
uint8_t	currentBPM	現在演奏中の BPM データ	MotorControl	1
float	RPM	BPM から計算された目標回転数	MotorControl	4
float	omega	目標角速度 ω	MotorControl	4
uint8_t	pwmValue	analogWrite 用の出力値	MotorControl	1
float	V_average	平均電圧 $V_{average}$	MotorControl	4

関数の仕様 本設計において、演奏制御および同期処理を実現するために実装する関数を表 4 に示す。

表 4 実装する関数の一覧表.

戻り値	関数名	引数	概要	所属
void	receiveUDP	なし	指揮デバイスから送信される BPM の段階番号を受信する. 受信されたデータは currentBPM に格納する	SyncManager
void	startRamp	なし	演奏開始時の加速を制御する. 予約された BPM に基づき 1 小節分の時間をかけて徐々に目標速度 (RPM) へ到達させる.	MotorControl
void	changeBPM	uint8_t	演奏中の BPM 変更を行う.	MotorControl

今後の説明では, 表 4 で定義した関数を用いて処理ロジックを述べる.

Arduino のメモリの検証 本設計がターゲットハードウェアのメモリ制約を満たすか検証を行う. 使用するマイコンボードは Arduino UNO R4 WiFi である. データシートによれば, 本機に搭載されているマイコンは, プログラムの実行中に変数や計算結果を保持する SRAM を 32 KB, プログラム本体を保存する Flash メモリを 256 KB 備えている [2].

表 1 表および 3 に示した通り, 本システムのメモリ使用量は 123 byte である. これは SRAM 全体の約 0.3% 程度に留まっており, 無線通信ライブラリや制御スタックによる動的なメモリ消費を考慮しても, 十分な容量を確保できていると判断した.

2 通信機能の設計

2.1 全体の概要

本設計では, Wifi を用いた UDP による通信と, 物理同期 LED を用いた同期を行っている. 通信の概要を図 1 に示す.

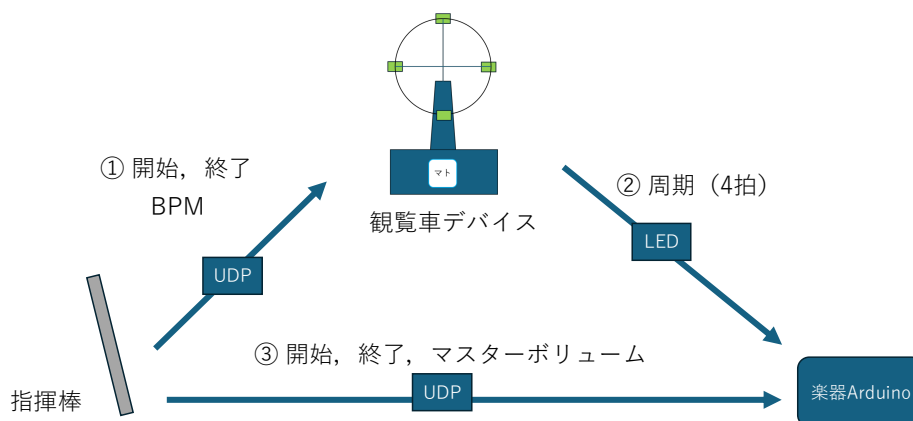


図 1 本システムの通信経路の概要.

2.2 無線通信仕様 (UDP)

無線通信は、利用するマイコンである Arduino UNO R4 WiFi に合わせるため WiFiS3.h ライブラリを使用する。指揮デバイス (指揮棒) を送信元とし、中継機へは BPM の段階番号を、楽器へはタイミングをずらしながら開始信号 (ヘッダ) と音量をユニキャストで個別に送信する仕様とする。なお、終了時の演奏終了ヘッダは両方に送信する。指揮デバイスに搭載された可変抵抗器 (BPM, 音量をユーザーが操作する用) および指揮棒 (加速度センサを搭載した開始・終了検知用) の状態に基づき、表 5 のように電圧値から変換された 1~5 の段階番号や制御コマンドを受信する。

表 5 段階と入力値範囲の対応表.

段階	入力値の範囲
1	0 – 204
2	205 – 409
3	410 – 614
4	615 – 819
5	820 – 1023

受信した段階番号を元に、中継機および楽器デバイスは表 6 のような変換をする。

表 6 各段階に対応する BPM および音量設定値.

段階	BPM 設定値	音量設定値
1	60	0
2	90	64
3	120	128
4	150	192
5	180	255

パケットの構造は表 7、表 8 のように、識別用のヘッダとデータ本体からなる合計 2 バイトの固定長ペイロードとして定義する。

表 7 通信の識別ヘッダ. (第 1 バイト:char)

ヘッダ	値 (ASCII コード)	内容
“B”	0x42	BPM データ
“V”	0x56	音量データ
“S”	0x53	演奏開始 (Start)
“E”	0x45	演奏終了 (End)

表 8 各ヘッダに対応したペイロードの内容. (第 2 バイト:uint8_t)

ヘッダ	ペイロード内容
“B”	BPM の段階番号 (1~5) を受信. 中継機内部の bpmTable[] を参照し, 設定値へ変換.
“V”	音量の段階番号 (1~5) を受信. 楽器ユニットへ送信される.
“S”	演奏開始コマンド. 各楽器ユニットが受信し, このパケットの到達をトリガーとして即座に演奏を開始.
“E”	演奏終了コマンド. 第 2 バイトにはダミーデータ (0) が格納されており, 受信後は停止処理へ移行.

UDP は到達保証がないプロトコルなので, システムの信頼性を高めるために次のような設計を採用する.

- 冗長送信: パケットロスによる制御の失敗を防ぐため, BPM 変更, 演奏開始・終了の各イベント発生時に, 短い間隔 (例えば 20 ms) で同一パケットを 3 回連続して送信する仕様とする.
- 送信タイミング: 指揮デバイスの状態 (BPM・音量の段階) が変化した瞬間に即時送信を行う. 状態に変化がない間はパケット送信を行わないことで, ネットワーク帯域の占有を最小限に抑える.

ここで 20 ms は, BPM120 の 1 拍 (500ms) に対して十分短い時間なため, リズムのズレとして許容できる.

2.3 物理同期信号の設計 (LED)

各カゴに搭載された LED は常に点灯される. 観覧車の回転に伴って, LED が楽器側の照度センサーを通過する際の光の立ち上がりを物理的な同期パルスとして利用する. 本設計では 1 象限の移動を 1 小節と定義しているため, このパルス検知は各小節の第 1 拍の開始を意味している.

楽器ユニット側は, 演奏開始前の空転時間において, このパルスの検出間隔から事前に実際の BPM を計算して演奏待機状態とする. 実際の演奏開始は, 指揮デバイスから個別に送信される演奏開始パケット (ヘッダ: 'S') を受信した瞬間をトリガーとする. 2 回目以降のパルス検知は, 物理的な回転速度のズレを補正するための基準信号として利用する設計とする.

3 ソフトウェア設計

3.1 ファイル構成

各役割ごとに, ファイルを次のように定義する.

- FerrisWheel.ino: メインロジックを担当する. loop 関数における UDP 受信処理や変速処理の呼び出しなどを行う.
- MotorControl.cpp: モーターの制御を行う. ルックアップテーブルに基づく回転速度の切り替えや, 起動時の加速を担当する.

- MotorControl.h:MotorControl.cpp のヘッダーファイルを定義する.
- SyncManager.cpp: 通信を行う. 指揮デバイスからの UDP パケットの受信処理を行う.
- SyncManager.h:SyncManager.cpp のヘッダーファイルを定義する.

3.2 オブジェクト設計

3.2.1 クラス図

本設計におけるクラス図を図2のように定義する.

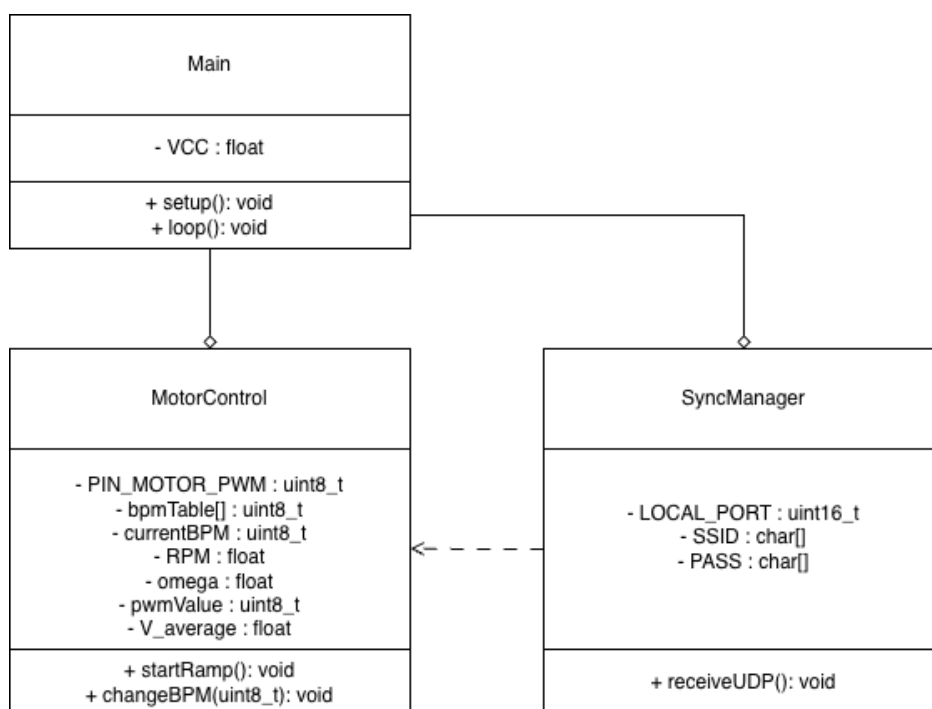


図2 本システムのクラス図.

メインとなる Main クラスが全体の進行を管理し, モータ制御を担う MotorControl クラスと通信・同期を担う SyncManager クラスを集約 (所有) する構成とした. これにより, 各機能の独立性を高めつつ, システム全体の司令を明確にしている.

4 処理フローの設計

4.1 LED を用いた演奏同期機能設計

ここでは, 楽器の目の前をカゴが通過する瞬間に, 正確に同期信号を楽器に送ることを目的とする. 本機能は, 通信クラスである SyncManager によって管理される.

同期のメカニズム 本設計では、観覧車の電源に直接接続された、土台側に配置された4つのLEDを常に点灯させておく。これに対して、楽器ユニットは固定されているため、観覧車の回転運動によってLEDが照度センサの正面を横切る瞬間のみ、センサ側で受光エネルギーの変化（物理パルス）が観測される。この一連の仕組みを図3に示す。

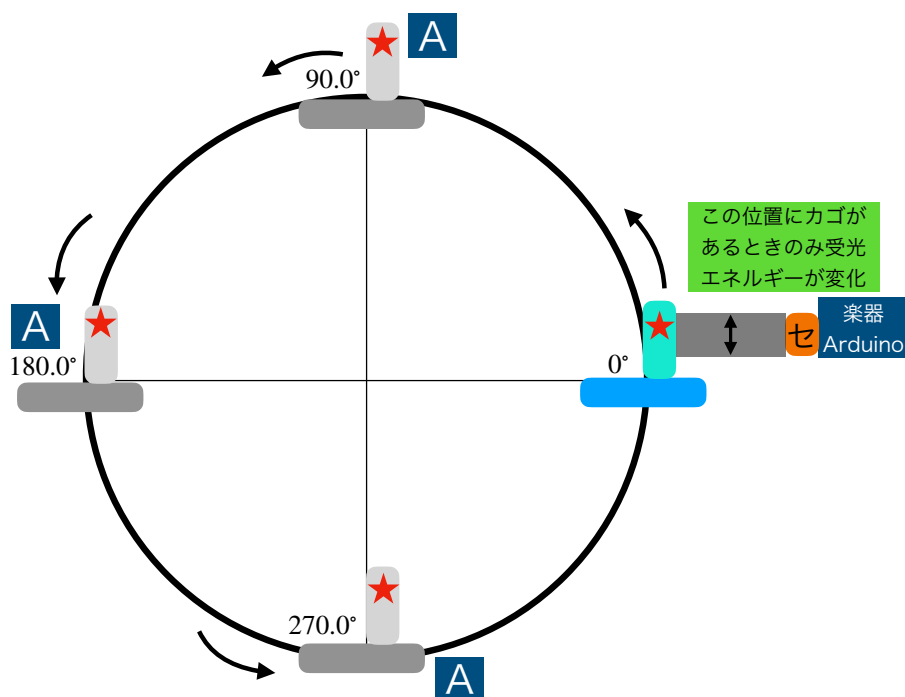


図3 同期システムの概念図。

4拍子の曲において1小節で1象限（90°）回転するという条件では、各楽器が1拍目の基準を検知する物理的な角度は表9の通りである。

表9 各LED（センサ）の配置角度。

LED 番号	配置角度
LED1	0°
LED2	90°
LED3	180°
LED4	270°

これにより、演算や通信の遅延に影響されず、物理現象に基づく確実な同期を実現する。楽器ユニット側は、演奏開始前の時間にこのパルスが検出される時間間隔（周期）から現在のBPMを自動で逆算し、演奏待機状態とする。なお、演奏開始後における2拍目以降のタイミングについては、楽器側でこの物理パルスを基準に内部Tickを補正し、生成する仕様となっている。

4.2 回転速度同期機構の設計

モータ制御の原理と BPM の変換 ここでは、BPM という数字を、実際の回転数 RPM に変換させることを目的とする。本システムでは、Arduino の PWM (パルス幅変調) 機能を用いてモータの平均印加電圧を制御する。供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。

$$V_{average} = V_{cc} \times \frac{pwmValue}{256} \quad (1)$$

また、BPM をモータの目標回転数 RPM へ変換するロジックを定義する。本システムでは、4 拍で 90 度回転する仕様に基つき、RPM は (2) 式で表せ、1 回転の時間は (3) 式で求められる。

$$RPM = \frac{BPM}{16} \quad (2)$$

$$60 \div RPM = 60 \div \frac{BPM}{16} = \frac{960}{BPM} \quad (3)$$

制御の指標となる角速度 ω は (4) 式で求められる。

$$\omega = 360^\circ \div \frac{960}{BPM} = \frac{3}{8} BPM \quad (4)$$

ルックアップテーブルによる速度制御 一般的な DC モータの回転速度は供給電圧に比例する性質があるが [3]、実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い。そのため近似直線を用いる手法ではなく、より確実な速度追従を実現するため、実測値に基づくルックアップテーブル方式を採用する。

MotorControl クラスは、表 10 に示す離散的な 5 段階の BPM 設定値を保持し、受信した段階番号に応じた $pwmValue$ を即座に選択・出力する構成とする。

表 10 BPM に対する目標 RPM と実測出力 PWM 値

段階	目標 BPM	目標 RPM	出力 PWM 値
1	60	3.75	(実測値 A)
2	90	5.625	(実測値 B)
3	120	7.5	(実測値 C)
4	150	9.375	(実測値 D)
5	180	11.25	(実測値 E)

目標 RPM は (2) 式から算出している。ここからは実測値の測定方法について説明する。ただし、このロジックはシステムに本実装するプログラムではなく、測定方法であることに注意する。まず、1 象限分回る時間 (T_{target}) を (5) 式で計算する。

$$T_{target} = \frac{60}{BPM} \times 4 \quad (5)$$

そこから、任意の PWM 値でモーターを回し、実験用の外部計測器等を用いてカゴが 1 象限を通過する時間を `millis()` 関数等を用いて測定する。この測定時間が T_{target} に限りなく近づくまで PWM 値を微調整する。

4.3 演奏制御機能設計

開始時の動作 ここでは、音楽と回転の同時スタートを管理することを目的とする。具体的には、指揮デバイスから BPM の段階番号を受信したあと、観覧車が目標速度で安定空転する時間を設け、楽器側に BPM を測定させる設計としている。

本ロジックは、観覧車の電源が投入され、かつ指揮デバイスから BPM の段階番号を受信した際に行われる。まず、受信時の動作を図 4 に示す。

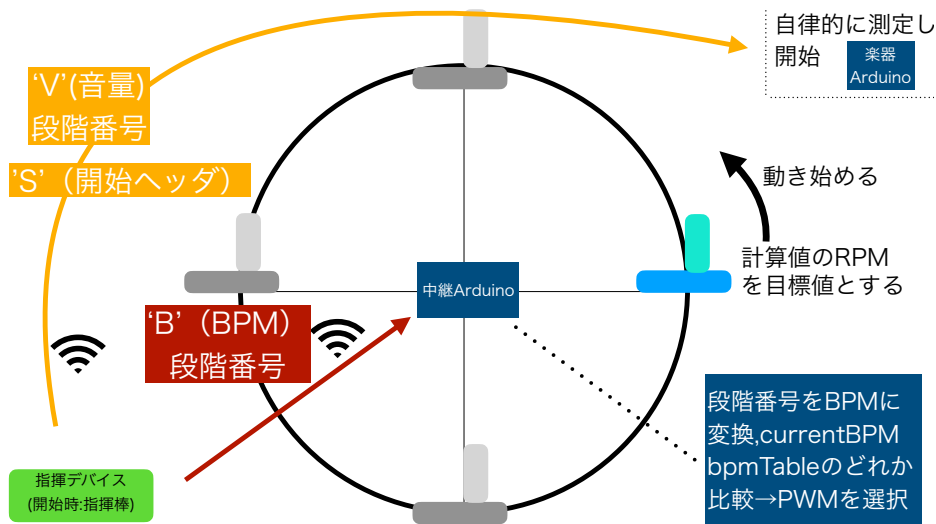


図 4 指揮信号を受信したときの動作と目標 RPM への加速フェーズ。

指揮デバイスから受信された BPM の段階番号は、`SyncManager::receiveUDP()` を経て、`MotorControl` クラスへと引き渡される。`MotorControl` は、クラス内部の `bpmTable[]` から該当する実測 PWM 値を選択し、これをモータの初期速度とする。

観覧車を静止状態から目標速度へ一気に上げると負荷が大きく、リズムの乱れが生じる可能性がある。そのため、本設計では `MotorControl::startRamp()` を用いて 1 小節分 (4 拍分) の時間を加速のための猶予期間として確保する。この猶予の間に `MotorControl` は目標とする安定した回転速度 (RPM) へと到達させるために段階的に加速させる。

観覧車が目標速度に到達したら、その速度で回転を維持する。この期間に、常時点灯している土台の LED パルスを楽器ユニット側が検知することで、楽器側が自律的に BPM を事前測定するための時間を確保する。楽器側の測定が完了したときに、指揮デバイス (指揮棒) 側の演奏開始操作をトリガーとして合奏を開始する。指揮デバイス側は、カゴが各楽器の正面に届くまでの時間を自身のタイマーから逆算することでモータが安定走行に達した状態と判断できる。そして、各楽器の物

理的な配置に合わせて適切な時間差をはさみ演奏開始信号と音量をユニキャストで個別に直接送信する。各楽器ユニットは、このユニキャストによる開始信号を受信した瞬間をトリガーとして一音を鳴らすことで、正確な発音を開始する設計とする。

BPM (音量) 変更設計 ここでは、演奏中に指揮デバイスから新しい BPM 段階番号を受信した場合のロジックを説明する。本システムでは、実際の回転速度（光パルスの間隔）そのものを BPM の基準としているため、観覧車側は受信した段階番号に応じて即時に回転速度を変更する仕様とする。具体的には、MotorControl クラスがルックアップテーブルから対応する pwmValue を読み取り、モーターへ反映させる。

本設計では、楽器ユニット側が LED の物理パルスからリアルタイムに BPM を逆算する構成となっている。そのため、観覧車側の速度が変更された場合でも、楽器側が n 拍目の光を検知した時刻 T_n とその直前の時刻 T_{n-1} の間隔 $\Delta T = T_n - T_{n-1}$ を用いて、実際の演奏 BPM (BPM_d) を次式のように算出し、自動的に新しい速度へ追従・同期できる仕様とする。

$$BPM_d = \frac{60000}{\Delta T} \quad (6)$$

5 テストの設計

5.1 BPM 追従テスト

ここでは、目標とする BPM に対して実際の回転速度がどの程度追従しているかを検証することを目的とする。具体的には、指揮デバイスから各段階の BPM を送信し、観覧車が各目標速度で安定して回転するかを確認する。

- 方法: MotorControl クラスがルックアップテーブルから pwmValue を選択し出力する。楽器ユニット側の照度センサがパルスを検知した時間間隔から算出される実測 BPM_d が、目標値に対して許容誤差以内であることを確認する。
- 合格基準: すべての BPM において、目標値の $\pm 5\%$ 以内であること。

許容誤差の選定にあたっては、聴取者が違和感を抱かない範囲を基準とした。参考文献 [4] によると、単発の音の間隔変化における検知閾値 (JND) は平均約 6% であるが、連続する規則的なシーケンスでは感度が高まり約 3% となる。また、非音楽家を対象とした実験では、等間隔のシーケンスに対する JND の平均値は 5.6% と報告されている。したがって、非音楽家を主なターゲット層と想定する場合、 $\pm 5\%$ 程度がほとんどの人が変化に気づかない妥当な合格基準であると考え、設定した。

5.2 同期精度テスト

ここでは、物理的な動きと楽器の発音タイミングがどの程度一致しているかを検証することを目的とする。具体的には、観覧車の LED が照度センサを通過する物理的なタイミングと、楽器側の発音タイミングが一致し、許容範囲内の遅延におさまっているかを確認する。

- 方法: LED の通過時刻と、楽器ユニットが音声を出力したタイミングの差分をオシロスコープ等で測定する。
- 合格基準: LED 検知から発音までの遅延が 50 ms 以内であること。

各楽器の発音が一体として知覚されるためには、同期のズレを許容範囲内に抑える必要がある。先行研究 [5] では、意図しない離散的反射音（エコー）として認識される遅延は 50 ms を超える場合に発生することが示されている。この知見に基づき、本システムでは通信・処理の合計遅延時間を 50 ms 以内に収めることを設計目標とした。

5.3 負荷テスト

ここでは、システムに高負荷をかけた際の挙動の安定性を検証する。

- 検証 1（通信負荷）：BPM や音量の変更パケットを短時間で連続してユニキャストし、`SyncManager::receiveUDP()` における通信の取りこぼしやフリーズが発生しないかを確認する。
- 検証 2（物理負荷）：最大速度（180 BPM）で長時間連続運転を行い、モータなどの過熱による影響を確認する。
- 合格基準: 高負荷状態においても、システムが停止したり制御不能に陥ったりしないこと。

参考文献

- [1] Arduino WiFi Library, `wl_definitions.h`, https://github.com/arduino-libraries/WiFi/blob/master/src/utility/wl_definitions.h, (参照 2026-05-08).
- [2] Arduino, "Arduino UNO R4 WiFi Datasheet", <https://docs.arduino.cc/resources/datasheets/ABX00087-datasheet.pdf>, (参照 2026-04-28).
- [3] Control.com, "DC Motor Speed Control," *Lessons In Electric Circuits*, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, (参照 2026-05-04).
- [4] C. Drake and M. C. Botte, "Tempo sensitivity in auditory sequences: Evidence for a multiple-look model," *Perception & Psychophysics*, vol. 54, no. 3, pp. 277–286, 1993.
- [5] H. Haas, "Über den Einfluss eines Einfachechos auf die Hörsamkeit von Sprache," *Acustica*, vol. 1, pp. 49–58, 1951.